A test for the error you actually care about.

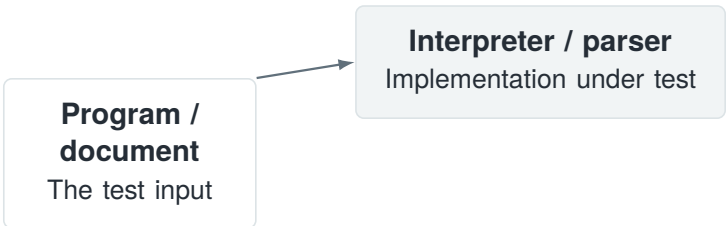
RuSMT

An Executable Semantics as Conformance Oracle and Test Suite Synthesizer

Mehrad Haghshenas **Meng Xu** (presenter)
University of Waterloo / SpecOps '26

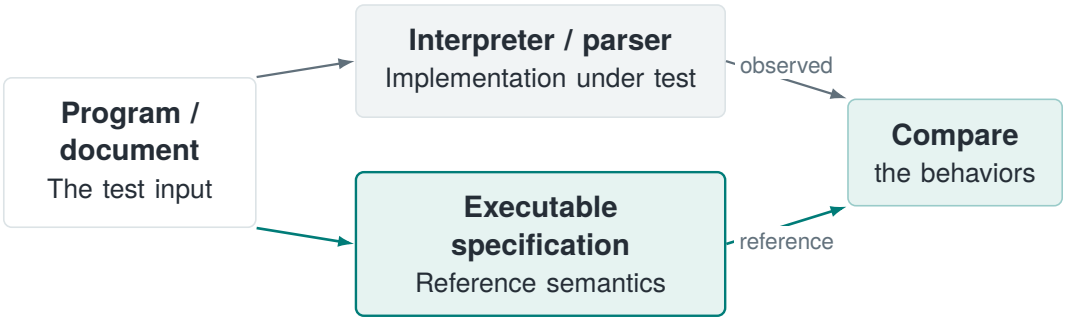
Name the condition. Generate its test.

Test the interpreter. Supply a program.



The program or document is the **test input**.

Test the interpreter. Supply a program.



The program or document is the **test input**.

A disagreement is something to investigate—in the implementation or the formalization.

The starting point was a Rego interpreter.

Regorus

Microsoft's Rego interpreter written in Rust.

github.com/microsoft/regorus

Which **language behaviors** have we exercised?

Not just which lines of the implementation happened to execute.

Rego motivated the question. IMP and TOML are the case studies.

No Rego formalization is evaluated in this paper.

These are different testing requests.

“Give me an invalid input.”

These are different testing requests.

“Give me an invalid input.”

“Give me an input that reaches **this particular error condition**.”

The second request specifies what the test should exercise.

Both fail. Only one tests the requested condition.

TARGET Division by zero

:= means assignment. The initial store has no variable bindings.

v0 := v0

Undefined variable

Different condition

Both fail. Only one tests the requested condition.

TARGET Division by zero

`:=` means assignment. The initial store has no variable bindings.

`v0 := v0`

Undefined variable

Different condition

`A := (0 / 0)`

Division by zero

Requested condition

An expected language error is not an interpreter crash.

State what you want exercised.

Coverage tells us
what we exercised.

A marker lets us state
what we want exercised.

In RuSMT, that target lives inside the executable specification.

Represent the IMP syntax as data.

ARITHMETIC EXPRESSIONS

```
#[smt_type]
pub enum Aexp {
    Num(I64),
    Var(String),
    // other operators omitted
    Div(Cloak<Aexp>,
        Cloak<Aexp>),
}
```

COMMANDS

```
#[smt_type]
pub enum Com {
    Skip,
    Assign(String, Cloak<Aexp>),
    // Seq, If, While omitted
}
```

`#[smt_type]` marks a DSL type.

`Cloak<T>`: Box-like indirection for recursive types.

IMP: 1,084 nonblank lines. Only selected variants are shown here.

The entry point fixes the store, not the program.

RESULT OF THE INTERPRETER

```
#[smt_type]
pub enum EvalResult {
    Err(Path),
    Ok(Array<String, I64>),
}
```

A final store, or a modeled error.

ENTRY FUNCTION

```
#[smt_fn]
pub fn eval_com(c: Com)
    -> EvalResult {
    let s = Array::new();
    eval_com_inner(c, s)
}
```

Start with no variable bindings.

One free input: `c`, the IMP program.

Assignment evaluates its expression first.

Inside `#[smt_fn] eval_com_inner`; other command cases omitted.

```
Com::Assign(x, a) => {
    let v = eval_aexp(a.reveal(), s);
    match v {
        ArithmeticResult::Err(e) => EvalResult::Err(e),
        ArithmeticResult::Ok(new_v) =>
            EvalResult::Ok(s.store(x, new_v)),
    }
}
```

Return the expression's error, or return the updated store.

Return the error. Name the condition.

Inside `eval_aexp`, after both division operands return values.

```
if *new_r.eq(I64::from(0)) {
  ArithmeticResult::Err(Path::named("division_by_zero"))
} else {
  ArithmeticResult::Ok(new_l.bv_div(new_r))
}
```

Guard the primitive.
Return the language's modeled error.

Explicit error semantics, not a proof of totality or panic-freedom.

Return the error. Name the condition.

Inside `eval_aexp`, after both division operands return values.

```
if *new_r.eq(I64::from(0)) {
  ArithmeticResult::Err(Path::named("division_by_zero"))
} else {
  ArithmeticResult::Ok(new_l.bv_div(new_r))
}
```

Guard the primitive.
Return the language's modeled error.

Name this condition.
The branch is also a testing request.

Explicit error semantics, not a proof of totality or panic-freedom.

The other marker comes from reading the store.

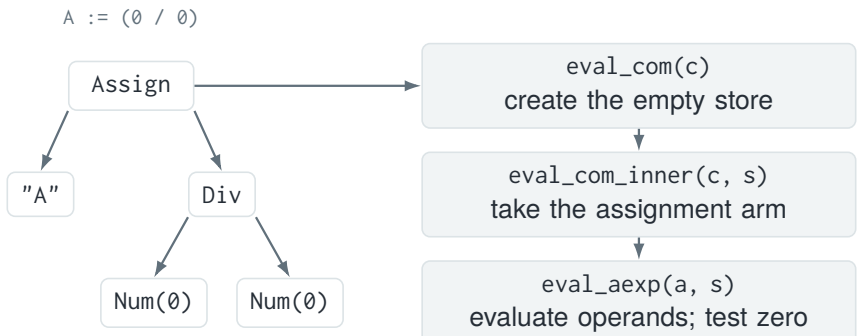
Inside `#[smt_fn] store_get(s, x)`.

```
if *s.contains_key(x) {
    StoreGetResult::Ok(s.select(x))
} else {
    StoreGetResult::Err(Path::named(
        String::from("undefined_variable")))
}
```

`v0 := v0` ⟶ read `v0` from the empty store.

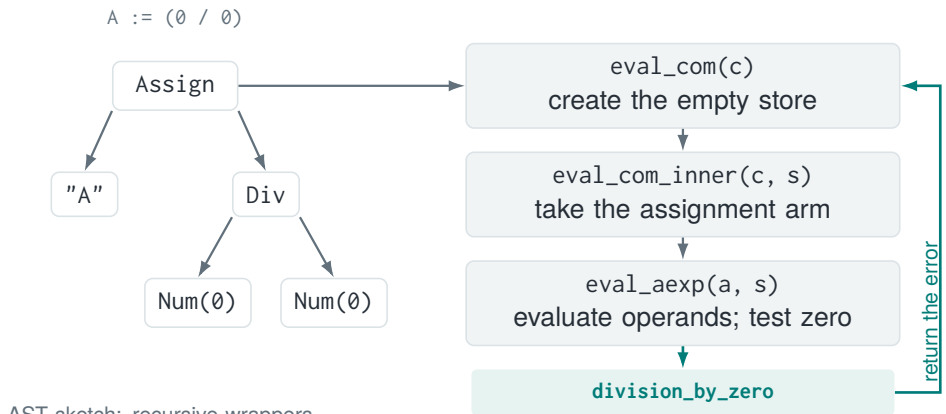
No binding found: `undefined_variable`

Trace the program through the evaluator.



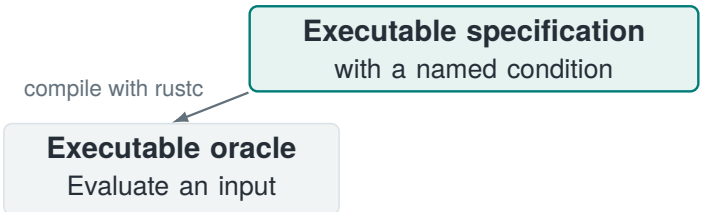
AST sketch: recursive wrappers and namespaces omitted.

Trace the program through the evaluator.



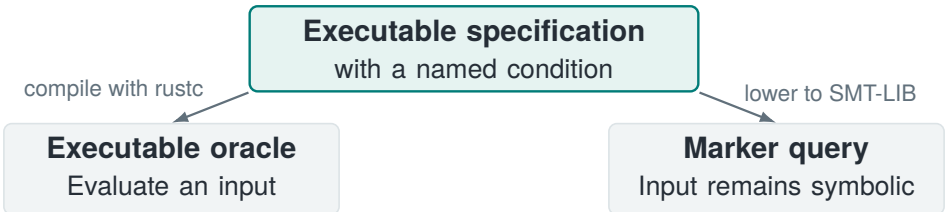
AST sketch: recursive wrappers and namespaces omitted.

Keep the evaluator fixed. Find its input.



The evaluator and initial store stay fixed. **Only the input program is unknown.**

Keep the evaluator fixed. Find its input.



The evaluator and initial store stay fixed. **Only the input program is unknown.**

Both IMP tests come from this evaluator.

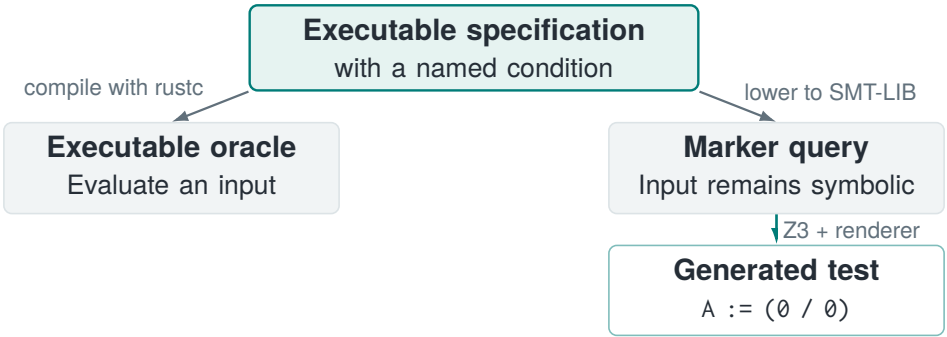
Unknown `c` ⟶ `eval_com(c)` returns the requested marker.

Target in the evaluator	Rendered test	Z3 time
`division_by_zero`	`A := (0 / 0)`	220 ms
`undefined_variable`	`v0 := v0`	109 ms

2 / 2 markers reached. Native recursion. No language model.

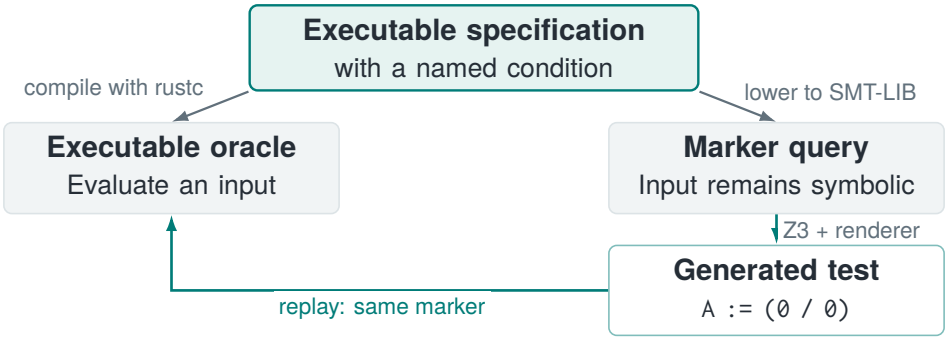
IMP is the synthesis-and-replay demonstration, not the differential parser experiment.

Generate it. Then replay the same condition.



The renderer produces source text; concrete replay must reach the requested marker.

Generate it. Then replay the same condition.



The renderer produces source text; concrete replay must reach the requested marker.

Now try a complete TOML parser.

183

named conditions

5,878 nonblank lines

Strings and escapes

Numbers and datetimes

Arrays, keys, and tables

Syntax errors and semantic violations,
including duplicate keys and redefined tables.

A hand-written executable formalization of the published specification.

Two opening brackets. Two closing brackets.



An array-of-tables header.

The closing delimiter is a **pair** of brackets.

First try the complete closing delimiter.

The key has been parsed. `state_after_key` is the saved position just after it.

```
match parse_array_table_close(state_after_key) {
    ParseResult::Ok(_close_brackets, state_after_close) =>
        return ParseResult::Ok(key, state_after_close),
    ParseResult::NoMatch => match current_char(state_after_key) {
        // the two error arms are expanded next
    },
    ParseResult::Err(e) => return ParseResult::Err(e),
}
```

The helper recognizes whitespace + `]]`; `NoMatch` is not yet a named error.

The saved cursor selects the error.

Both documents fail to match `]]`; inspect `state_after_key`.

```
match current_char(state_after_key) {
    Optional::Some(_ch) =>
        return ParseResult::Err(Path::named(String::from(
            "array_table_missing_close_char"))),
    Optional::None =>
        return ParseResult::Err(Path::named(String::from(
            "array_table_missing_close_eof"))),
}
```



→ None / EOF

The colored bar is the saved cursor, not part of the input. Arm braces elided for layout.

The saved cursor selects the error.

Both documents fail to match `]]`; inspect `state_after_key`.

```
match current_char(state_after_key) {
    Optional::Some(_ch) =>
        return ParseResult::Err(Path::named(String::from(
            "array_table_missing_close_char"))),
    Optional::None =>
        return ParseResult::Err(Path::named(String::from(
            "array_table_missing_close_eof"))),
}
```

`[[a|` → None / EOF `[[a|]x` → Some / character

The colored bar is the saved cursor, not part of the input. Arm braces elided for layout.

Direct synthesis stalls on the real parser.

IMP

2/2

markers reached

TOML

0/183

markers reached

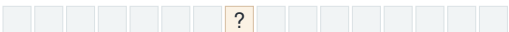
TOML: no decisive result within the experimental budget.

No witness found is **not** a proof that the target is unreachable.

Fixing the input changes the cost sharply.

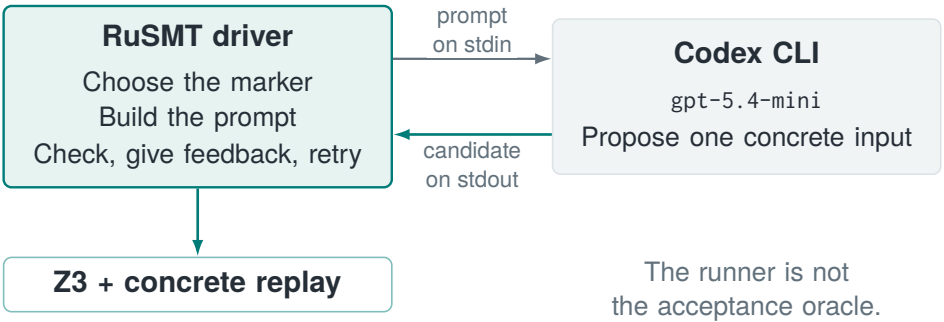
All 16 characters fixed — **798 ms**

One character left free — **unknown at 30 s**



Checking a proposed input can be easier than finding one.

Codex proposes. RuSMT runs the loop.



A subprocess interface, not a direct model-API integration in the RuSMT driver.

What goes into one prompt?

Task Object language: `toml`
Reach `array_table_missing_close_char`.

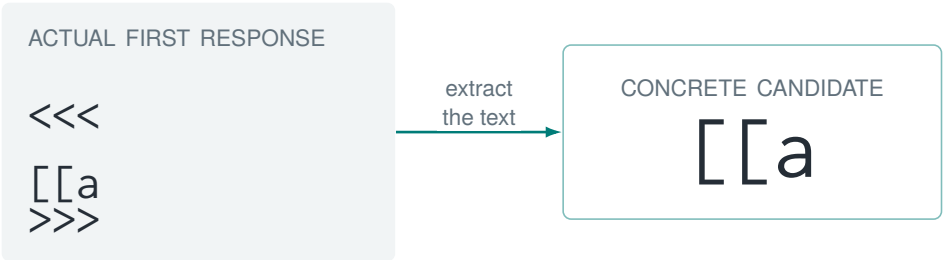
Solver context Emitted SMT-LIB for this marker
The recorded prompt supplies an excerpt; not the Rust source shown earlier.

Retry context Earlier candidates + Z3 verdicts
Including the marker actually reached, when diagnosis succeeds.

Hints + response format
Only the first error matters. Keep it short. Return one delimited input.

Hints guide the search. The original marker query decides success.

One document comes back, not a solver model.



RuSMT fixes the original query's input to this document.

The response is not an interpreter patch, a rewritten query, or a correctness verdict.

What did the rejected input actually do?

TARGET: WRONG CLOSING CHARACTER

Candidate: `[[a`

1 Target check Keep the original target; fix this input.

unsat This document does not reach the requested marker.

What did the rejected input actually do?

TARGET: WRONG CLOSING CHARACTER

Candidate: `[[a`

1 Target check Keep the original target; fix this input.

unsat This document does not reach the requested marker.

2 Diagnostic query Same input; remove the target assertion.

Returned marker: `array_table_missing_close_eof`

Feedback: a different error fired first. Fix the earlier violation.

The next prompt includes the failed attempt.

Same goal:
`array_table_missing_close_char`

`[same SMT-LIB context]`

Candidates already rejected:

```
attempt 1: [[a
Z3: unsat
actually reached:
  array_table_missing_close_eof
```

`Reply: input <<< candidate >>>`

Prompt contents condensed; feedback wording shortened. No Rust-source edit is requested.

The next prompt includes the failed attempt.

Same goal:
`array_table_missing_close_char`

`[same SMT-LIB context]`

Candidates already rejected:

```
attempt 1: [[a
Z3: unsat
actually reached:
  array_table_missing_close_eof
```

`Reply: input <<< candidate >>>`

ACTUAL NEXT RESPONSE

`<<<[[a]x>>>`

The candidate changes.

The target stays fixed.

Prompt contents condensed; feedback wording shortened. No Rust-source edit is requested.

The target check and replay agree.

TARGET: WRONG CLOSING CHARACTER
`array_table_missing_close_char`

PROPOSED DOCUMENT

```
[[a]x
```

Document status
Rejected by oracle

OBSERVED BEHAVIOR

Wrong closing character

TARGET REACHED

1 **Z3: target check passed**
Original query, with this input fixed.

The target check and replay agree.

TARGET: WRONG CLOSING CHARACTER
`array_table_missing_close_char`

PROPOSED DOCUMENT

```
[[a]x
```

Document status
Rejected by oracle

OBSERVED BEHAVIOR

Wrong closing character

TARGET REACHED

1 **Z3: target check passed**
Original query, with this input fixed.

2 **Replay: same marker**
Compiled semantics confirms the result.

One witness requested. Nine rounds allowed.

146

of 183 markers reached

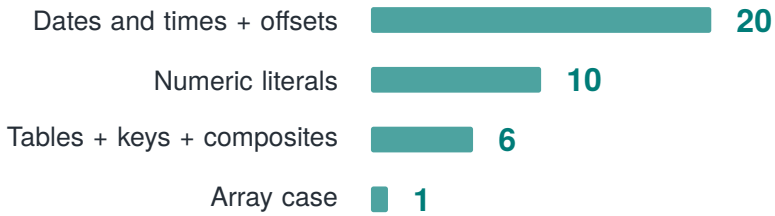
37 remain unresolved.
No hand-written fill-ins.

Stop after the first witness
or after **nine proposal rounds**.

Cumulative witnesses	
Round 1	57
Round 2	113
Round 9	146

TOML: 2 s unconstrained Z3 probe; 20 s for each pinned-candidate check.

Most misses are numeric or temporal.



Reaching a later numeric or temporal error may require passing many earlier checks.

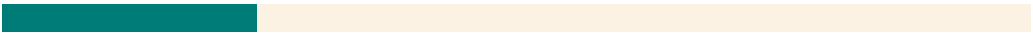
No witness within the budget is not a proof of unreachability.

About three rejected proposals per witness.

588 PROPOSALS REACHED Z3 AND RECEIVED A VERDICT

146
accepted for the requested target

442
`unsat`: rejected for that target



Separate from these checks: 28 repeated candidates and 19 replies with no parsable candidate.

The common miss is the wrong first error.

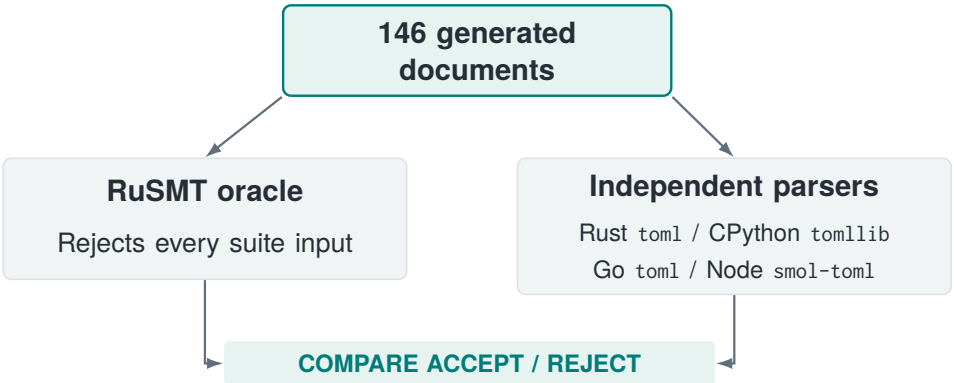
88.2%

of rejections were diagnosed as **reaching a different marker**.



Also: 5.9% valid TOML; 5.9% no diagnostic result. Percentages are of rejections.

One oracle. Four independent parsers.



A divergence here is a parser accepting a document rejected by the oracle.

Twelve disagreements. Not twelve bugs.

9

Permitted differences

The standard leaves the choice open.

2

Known defects

Already tracked upstream.

1

Previously unreported defect

Reported upstream.

A disagreement is a finding to investigate, not an automatic conformance verdict.

Our running example exposed a parser defect.

GENERATED WITNESS

```
[[a]x
```

MINIMIZED REPRODUCER

```
[[a]
```

smol-toml 1.8.0 accepts

For `[[a]`, produces:

```
{"a":[{}]}
```

The other three parsers reject

A closing bracket is missing.

Previously unreported at the time of the paper; filed upstream.

Related tests passed. This case was missed.

Existing unclosed-header cases

header	header	header	header	header
next line	next line	next line	next line	next line

Each continues on another line.

The minimized reproducer

```
[[a]
```

END OF INPUT

Passing related tests does not imply this case was exercised.

Some differences are permitted by the standard.

```
18446744073709551616
```

RuSMT's reference

Uses signed 64-bit integers.
Cannot represent this value; rejects it.

CPython's parser

Uses arbitrary-precision integers.
Can represent this value; accepts it.

Different behavior. Both can conform.

The formalization is still our responsibility.

Faithfulness is trusted.
Replay checks a witness, not the prose-to-code translation.

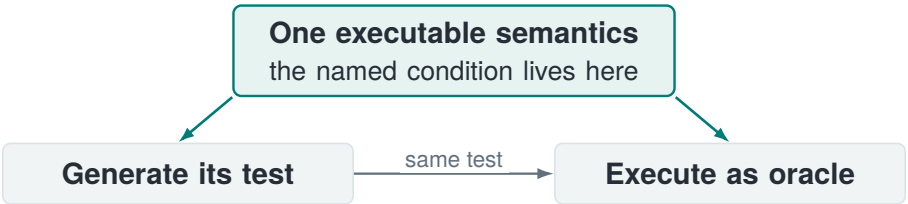
Coverage is marker-relative.
Unmarked conditions are not requested targets.

A miss is not unreachability.
The 37 uncovered TOML markers remain unresolved.

A targeted testing workflow—not a verified or complete formalization.

One condition. One source. Two uses.

Name the condition you want exercised.



146 / 183 TOML markers reached · 1 previously unreported defect

Mehrad Haghshenas and Meng Xu · meng.xu.cs@uwaterloo.ca

Explicit errors are not a totality guarantee.

What the DSL expresses

Modeled errors are returned values.

No explicit `panic!`, `?`, or `.unwrap()` in the fragment.

Guards define the target language's error behavior.

"Errors as values" is the paper's term.

What this does not prove

Every recursive execution terminates.

Every library call respects its precondition.

Every standard-mandated case is included.

What exactly does the target query ask?

For a selected set of markers P, find an input x such that:

$$\mathrm{entry}(x) = \mathrm{Err}(p) \quad \wedge \quad P \subseteq p$$

A proposed candidate c is checked by strengthening the original query:

$$Q_P \ \wedge \ (\mathrm{input}_0 = c)$$

Containment allows additional markers. A pinned `unsat` rejects that candidate,

not the reachability of the marker in general.

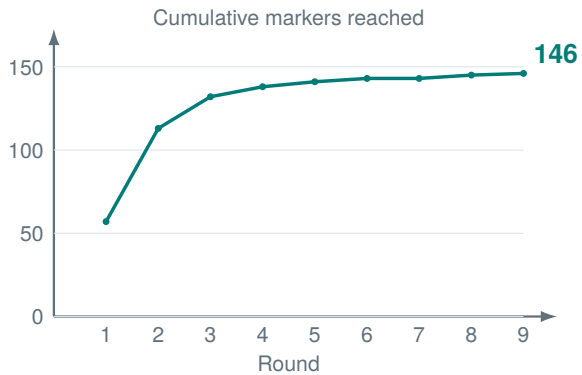
The same marker identity is used by the SMT encoding and compiled evaluator.

Experimental setup in the accepted paper.

Platform	Apple M1; macOS 26
Tools	Z3 4.15.4; Rust 1.88.0
Proposer	`gpt-5.4-mini`, invoked through the `codex` runner
Target budget	One witness per marker; nine rounds
Z3 budgets, TOML	2 s unconstrained probe; 20 s per pinned candidate
Encoding	Native recursion (`k=0`) in both case studies
Evidence	Recorded model exchanges and the generated suite

Reported coverage depends on the fixed computational budget; model reruns may differ.

Coverage arrives early; misses remain structured.



37 unresolved

20 date / time / offset

10 numeric literals

6 tables / keys / composites

1 array case

Agreement by implementation.

Implementation	Agreement	Divergences
Rust `toml` 1.1.4+spec-1.1.0	144/146	2
CPython `tomllib` (3.14.7)	137/146	9
Go BurntSushi `toml` v1.6.0	142/146	4
Node `smol-toml` 1.8.0	142/146	4

12 distinct inputs disagree; 19 parser–input pairs disagree.
Several parsers can disagree on the same input.

What is distinctive in this combination?

Rosette	A general solver-aided host; RuSMT packages the marked-semantics-to-suite workflow.
SCALE	Explores executable models through bounded scopes; RuSMT selects explicit marked error conditions.
JEST	Derives an oracle, synthesizer, and tester; RuSMT's test granularity is chosen by inline marker placement.

The semantics and its test requirements live in one source.

A tool-and-artifact contribution, not a new solver or a new propose-and-check principle.

Why retain Z3 for a fully concrete candidate?

Architecturally:

The candidate is checked against the **original symbolic obligation**.
The same query also supports direct synthesis, as IMP demonstrates.

Empirically:

The reported evaluation does not compare this validator with
a concrete-oracle-only validator.

Target checking matters. SMT is not shown to be uniquely necessary.

Native recursion, depth bounds, and the frontier.

Default: `k=0`	Native recursive definitions; used by both reported case studies.
Optional: `k>0`	Depth-indexed unrolling; a bounded `unsat` rules out only witnesses within that depth.
Resource limits	An `unknown` or timeout does not establish unreachability.
Future direction	Generate constraints along individual execution paths rather than lower the entire semantics into each query.

Do not confuse optional explicit depth unrolling with the reported default encoding.

BACKUP

The two defects that were already tracked.

EXTENDING AN INLINE TABLE

```
a = { b = 1 }
a.c = 2
```

Inline tables are self-contained.

REDEFINING A DOTTED-KEY TABLE

```
a.b = 1
[a]
```

The table already exists through the dotted key.

Accepted by the tested BurntSushi parser. Both issues were already tracked upstream.

The suite reached these cases from the executable specification, not by copying corpus tests.

Runner and prompt details from the archive.

Recorded runner configuration

```
codex exec
 -m gpt-5.4-mini
 -c model_reasoning_effort="low"
 -s read-only
 --skip-git-repo-check
 -o last.txt
```

Inside an OS sandbox denying source-tree reads; wrapper returns the final reply.

Prompt / response transport

Fresh working directory per invocation.

Explicit retry history in the prompt.

Candidate between <<< and >>>.

Prompt/response cache keyed by prompt hash.

These details describe the supplied runner; they do not establish which prompt or model setting caused a miss.